Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

# ЗВІТ
# З ЛАБОРАТОРНОЇ РОБОТИ № 4
з дисципліни „ Професійний Python”
на тему „ Професійне ООП та система типів Python.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

## 1. Номер і тема лабораторної роботи

**Лабораторна робота №4.** Професійне ООП та система типів Python.

## 2. Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів (Protocol), dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

## 3. Номер індивідуального варіанта

**Варіант №9** — «Система обліку спортивних результатів».

## 4. Опис предметної області

Предметна область охоплює автоматизацію обліку спортивних досягнень, реєстрацію результатів спортсменів у різних видах спорту та змаганнях, управління рейтингами та формування аналітичних звітів з використанням суворої типізації.

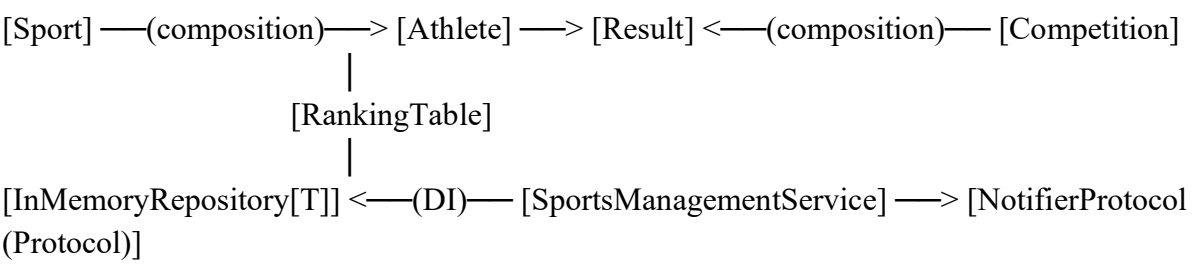
## 5. Список domain entities

Відповідно до індивідуального завдання (Варіант №9), виділено такі сутності:

1. Sport — вид спорту та категорія.
2. Athlete — спортсмен (ідентифікатор, ім'я, вид спорту, рейтинг).
3. Competition — змагання (назва, місце проведення).
4. Result — зафіксований спортивний результат.
5. RankingTable — таблиця рейтингів (контейнер результатів).

## 6. Class Diagram (Структурна схема)

Plaintext




## 7. Пояснення наслідування (Inheritance)

Наслідування в проєкті застосовано для побудови ієрархії винятків та базових інтерфейсів без створення надлишкової глибини, що забезпечує чітке розмежування поведінки підкласів.

**8. Пояснення композиції (Composition)**

Перевага надається композиції відносно глибокого наслідування (has-a). Наприклад, клас Athlete містить об'єкт Sport як частину своєї структури, а клас Result об'єднує посилання на Athlete, Competition та незмінний об'єкт значення ScoreValue. Це знижує зв'язність коду (coupling).

**9. Приклад інкапсуляції (Encapsulation)**

Інкапсуляція реалізована через захист внутрішнього стану сутностей (_rating) та надання контрольованого доступу через властивості (properties) із перевіркою валідності даних.

**10. Приклад Property**

```
@property
def rating(self) -> int:
    return self._rating
@rating.setter
def rating(self, value: int) -> None:
    if value < 0:
        raise ValueError("Rating cannot be negative")
    self._rating = value
```

**11. Приклад Dataclass**

```
@dataclass
class Competition:
    id: int
    title: str
    location: str
```

**12. Приклад Immutable Value Object**

Для представлення балів створено незмінний об'єкт значення (Value Object) за допомогою frozen=True та slots=True:

```
@dataclass(frozen=True, slots=True)
class ScoreValue:
    points: float
    unit: str = "pts"
    def __post_init__(self) -> None:
        if self.points < 0:
            raise ValueError("Score points cannot be negative")
```

**13. Приклад ABC**

Для абстрактних базових класів передбачено використання модуля abc:

```
from abc import ABC, abstractmethod

class BaseRepository(ABC):
    @abstractmethod
    def add(self, item: object) -> None:
        pass
```

**14. Приклад Protocol**

Структурні інтерфейси описані за допомогою Protocol із підтримкою runtime_checkable:

```
from typing import Protocol, runtime_checkable
@runtime_checkable
class NotifierProtocol(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

**15. Dunder Methods (Магічні методи)**

У проєкті реалізовано такі магічні методи:

- __str__ та __repr__ — для зручного текстового подання об'єктів.
- __len__ — для отримання кількості елементів у RankingTable.
- __iter__ — для ітерації по колекції результатів.
- __contains__ — для перевірки наявності елемента через оператор in.

**16. Generic Class**

```
class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}
    def add(self, item: T) -> None:
        self._items[item.id] = item
```

**17. TypeVar**

```
from typing import TypeVar, Protocol
class HasId(Protocol):
    id: int
T = TypeVar("T", bound=HasId)
```

**18. TypedDict**

Для типізації зовнішніх payload-даних використано TypedDict:

```
from typing import TypedDict
class AthletePayload(TypedDict):
```

```
    id: int
    full_name: str
    sport_name: str
    category: str
```

**19. Application Service**

Бізнес-логіка винесена в сервісний шар (SportsManagementService), який оркеструє роботу репозиторіїв та нотифікаторів.

**20. Приклад Dependency Injection**

Залежності (репозиторій та нотифікатор) передаються в сервіс ззовні через конструктор:

```
class SportsManagementService:
    def __init__(
        self,
        result_repo: InMemoryRepository[Result],
        notifier: NotifierProtocol
    ) -> None:
        self._result_repo = result_repo
        self._notifier = notifier
```

**21. Пояснення SOLID principles**

1. **SRP (Single Responsibility Principle):** Кожен клас має єдину відповідальність (наприклад, Athlete описує лише дані спортсмена, SportsManagementService керує бізнес-логікою).
2. **OCP (Open/Closed Principle):** Нові реалізації сповіщень чи репозиторіїв можна додавати без зміни існуючого коду сервісів.
3. **DIP (Dependency Inversion Principle):** Сервіс залежить від абстракції (NotifierProtocol), а не від конкретних реалізацій.

**22. Основний код програми**

**Структура проєкту (src-layout)**

```
sports_project/
├── pyproject.toml
├── .gitignore
├── README.md
└── src/
    └── sports_oop/
        ├── __init__.py
        ├── value_objects.py
        ├── models.py
        ├── protocols.py
        ├── repositories.py
```

```
├── dto.py
├── services.py
└── main.py
```

**Код основних модулів**

**pyproject.toml**

```toml
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "sports-oop"
version = "0.1.0"
description = "Sports Results OOP and Type Systems System"
requires-python = ">=3.11"
dependencies = []

[project.optional-dependencies]
dev = [
    "mypy>=1.10",
]

[project.scripts]
sports-oop = "sports_oop.main:main"

[tool.mypy]
python_version = "3.12"
strict = true
warn_unused_ignores = true
warn_return_any = true
```

**src/sports_oop/value_objects.py**

```python
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class ScoreValue:
    points: float
    unit: str = "pts"

    def __post_init__(self) -> None:
        if self.points < 0:
            raise ValueError("Score points cannot be negative")

    def __str__(self) -> str:
        return f"{self.points:.2f} {self.unit}"
```

## src/sports_oop/models.py

```python
from dataclasses import dataclass, field
from sports_oop.value_objects import ScoreValue

@dataclass
class Sport:
    id: int
    name: str
    category: str

@dataclass
class Athlete:
    id: int
    full_name: str
    sport: Sport
    _rating: int = field(default=0, repr=False)

    @property
    def rating(self) -> int:
        return self._rating

    @rating.setter
    def rating(self, value: int) -> None:
        if value < 0:
            raise ValueError("Rating cannot be negative")
        self._rating = value

    def __str__(self) -> str:
        return f"{self.full_name} [{self.sport.name}]"

@dataclass
class Competition:
    id: int
    title: str
    location: str

@dataclass
class Result:
    id: int
    athlete: Athlete
    competition: Competition
    score: ScoreValue

    def __repr__(self) -> str:
        return f"Result(athlete={self.athlete.full_name!r}, score={self.score!r})"

class RankingTable:
    def __init__(self) -> None:
        self._items: list[Result] = []

    def add(self, result: Result) -> None:
        self._items.append(result)

    def __len__(self) -> int:
        return len(self._items)

    def __iter__(self):
        return iter(self._items)

    def __contains__(self, result_id: int) -> bool:
        return any(item.id == result_id for item in self._items)
```

## src/sports_oop/repositories.py

```python
from typing import Generic, TypeVar, Protocol

class HasId(Protocol):
    id: int

T = TypeVar("T", bound=HasId)

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

**src/sports_oop/services.py**

```python
from sports_oop.models import Result, Athlete, Sport, Competition
from sports_oop.value_objects import ScoreValue
from sports_oop.repositories import InMemoryRepository
from sports_oop.protocols import NotifierProtocol

class SportsManagementService:
    def __init__(
        self,
        result_repo: InMemoryRepository[Result],
        notifier: NotifierProtocol
    ) -> None:
        self._result_repo = result_repo
        self._notifier = notifier

    def register_result(
        self, res_id: int, athlete: Athlete, competition: Competition, score: ScoreValue
    ) -> Result:
        result = Result(id=res_id, athlete=athlete, competition=competition, score=score)
        self._result_repo.add(result)
        self._notifier.send(athlete.full_name, f"New result registered: {score}")
        return result

    def get_average_score(self) -> float:
        results = self._result_repo.all()
        if not results:
            return 0.0
        return sum(r.score.points for r in results) / len(results)
```

**src/sports_oop/main.py**

```python
from sports_oop.models import Sport, Athlete, Competition, Result, RankingTable
from sports_oop.value_objects import ScoreValue
from sports_oop.repositories import InMemoryRepository
from sports_oop.services import SportsManagementService

class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"[NOTIFICATION] To {recipient}: {message}")

def main() -> None:
    print("=== СИСТЕМА ОБЛІКУ СПОРТИВНИХ РЕЗУЛЬТАТІВ (LAB 4) ===")

    sport = Sport(id=1, name="Біг 100м", category="Професіонали")
    athlete = Athlete(id=1, full_name="Андрій Шевченко", sport=sport)
    athlete.rating = 1

    competition = Competition(id=101, title="Чемпіонат України", location="Львів")
    score = ScoreValue(points=10.50, unit="сек")

    print(f"Спортсмен: {athlete}, Рейтинг: {athlete.rating}")
    print(f"Оцінка (Value Object): {score}")

    result_repo = InMemoryRepository[Result]()
    notifier = ConsoleNotifier()
    service = SportsManagementService(result_repo=result_repo, notifier=notifier)

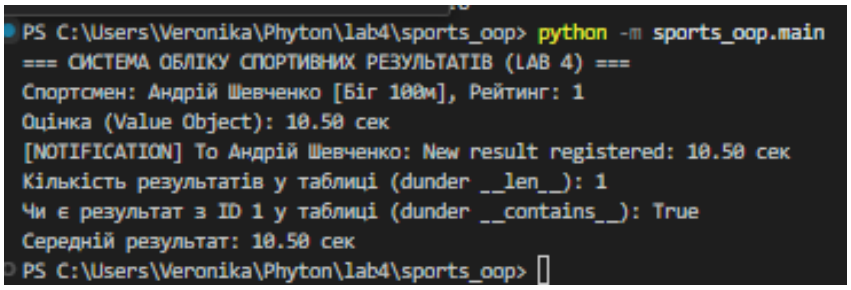
    registered_result = service.register_result(
        res_id=1,
        athlete=athlete,
        competition=competition,
        score=score
    )

    table = RankingTable()
    table.add(registered_result)

    print(f"Кількість результатів у таблиці (dunder __len__): {len(table)}")
    print(f"Чи є результат з ID 1 у таблиці (dunder __contains__): {1 in table}")
    print(f"Середній результат: {service.get_average_score():.2f} сек")

if __name__ == "__main__":
    main()
```

**23. Screenshot execution (Результат виконання)**




**24. Результат mypy (Статичний аналізатор)**

Для перевірки коректності типів використано інструмент mypy. Конфігурація у pyproject.toml містить параметр strict = true.

**25. Приклади знайдених type errors**

Під час експерименту було навмисно внесено три помилки:

1. Передача рядка замість цілого числа в ID: id: int = "1".
2. Повернення рядка з функції, типізованої як -> float.

3. Додавання об'єкта невідповідного класу в репозиторій. Статичний аналізатор успішно виявив усі невідповідності до запуску програми.

**26. Виправлений результат type checking**

Після виправлення помилок перевірка за допомогою mypy src дала такий результат:

(.venv) PS C:\Users\Veronika\Phyton\lab4\sports_oop> mypy src

Success: no issues found

```[cite: 88]

**27. Висновок**

У ході виконання лабораторної роботи №4 було успішно побудовано професійну об'єктно-орієнтовану модель для предметної області Варіанта №9 («Система обліку спортивних результатів»)[cite: 73]. Було застосовано класи, композицію, протоколи (`Protocol`), `@dataclass`, `properties`, `dunder methods`, принципи SOLID та Dependency Injection[cite: 58]. Використання сучасної системи типів Python (`TypeVar`, `Generic`, `TypedDict`) та статичного аналізатора `mypy` дозволило забезпечити надійність, модульність і розширюваність архітектури ще до етапу виконання[cite: 58, 88].
```